

OOP : Assignment 1B

1. Explain the concept of programming paradigms and discuss their importance in Python.

Programming paradigms are different ways to write and organize code. Python supports multiple paradigms like procedural, object-oriented, and functional. This makes Python flexible and suitable for a wide range of applications.

2. Compare Object-Oriented Programming (OOP) with Procedural Programming, highlighting key differences.

- OOP uses classes and objects, and focuses on *data and behavior together*.
 - Procedural programming uses functions and focuses on *step-by-step instructions*.
 - OOP supports inheritance, encapsulation, and reusability, which procedural does not.
-

3. Describe the key advantages of using the OOP paradigm in Python with suitable examples.

- Reusability (via inheritance)
- Modularity (code is organized in classes)
- Encapsulation (data protection)

Example:

```
class Car:
```

```
    def drive(self):  
        print("Driving")
```

4. Discuss how Python implements data hiding in classes and its significance.

Python uses name conventions:

- `_var`: Protected (for internal use)
- `__var`: Private (name-mangled)

Significance: It helps prevent accidental access to sensitive data.

5. Explain the concept of classes in Python and demonstrate how to define a class with relevant examples.

A class is a blueprint for creating objects.

class Person:

```
def __init__(self, name):  
    self.name = name  
  
def greet(self):  
    print(f"Hello, {self.name}")
```

6. Describe the steps involved in creating and initializing an object from a class in Python.

1. Define a class.
2. Use `__init__()` to initialize data.
3. Create an object using class name.

```
p = Person("John")  
p.greet()
```

7. What is the role of the self keyword in Python class methods? Explain with an example.

self refers to the current object. It allows access to attributes and methods.

```
class Dog:  
  
    def bark(self):  
        print("Woof")
```

8. Discuss the purpose and functionality of the `__init__()` method in Python classes with an example.

`__init__()` is the constructor. It initializes object attributes.

class Student:

```
def __init__(self, name):  
    self.name = name
```

9. Explain the concept of inheritance in Python. How does Python achieve inheritance in OOP?

Inheritance lets one class use features of another.

```
class Animal:  
    def sound(self): print("Sound")
```

```
class Dog(Animal): pass
```

Python uses class names in parentheses to achieve inheritance.

10. What is polymorphism in Python? Explain its role and provide suitable examples.

Polymorphism means using the same method name for different classes.

```
class Cat:  
    def speak(self): print("Meow")
```

```
class Dog:  
    def speak(self): print("Woof")
```

11. Define encapsulation in Python. How does it ensure data security in OOP?

Encapsulation means wrapping data and methods in one unit (class) and hiding internal details using private variables. It prevents direct access to sensitive data.

12. Explain abstraction in Python with an example. How does it improve code efficiency?

Abstraction hides complex implementation and shows only necessary parts.

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
```

```
    @abstractmethod
```

```
    def area(self): pass
```

It makes code cleaner and easier to maintain.

13. Discuss how Python handles memory allocation for objects. What mechanisms are used for efficient memory management?

Python uses:

- Automatic memory management
- Garbage collection
- Reference counting

It frees memory when objects are no longer used.

14. What is a constructor in Python? Explain its purpose and demonstrate how to define one.

A constructor initializes new objects. In Python, it's `__init__()`.

```
class Book:
```

```
    def __init__(self, title):
```

```
        self.title = title
```

15. Describe the significance of the `__del__()` method in Python. When is it called, and how does it work?

`__del__()` is a destructor. It's called when an object is about to be deleted.

Code:

```
def __del__(self):
```

```
    print("Object deleted")
```

16. What is the role of the super() function in Python? How is it used in inheritance?

super() calls methods from the parent class.

class A:

```
def __init__(self):  
    print("A init")
```

class B(A):

```
def __init__(self):  
    super().__init__()  
    print("B init")
```

17. Explain the difference between class attributes and instance attributes in Python. Provide suitable examples.

- Class attribute: Shared across all instances.
- Instance attribute: Unique to each object.

class Dog:

```
species = "Mammal"    # class attribute  
  
def __init__(self, name):  
    self.name = name  # instance attribute
```

18. Discuss the purpose of static methods in Python. How are they different from instance methods?

Static methods don't use self and don't access object data. Use @staticmethod.

class Math:

```
@staticmethod  
def add(x, y):  
    return x + y
```

19. What is a friend function in Python? How is it implemented, and what are its uses?

Python doesn't support friend functions like C++. But you can access private data using name mangling (`_ClassName__var`). It's not recommended; use public methods instead.

```
class MyClass:
```

```
    def __init__(self):
```

```
        self.__secret = 42
```

```
def friend_func(obj):
```

```
    print(obj._MyClass__secret) # Accessing private member
```

```
obj = MyClass()
```

```
friend_func(obj)
```

20. Explain the concept of arrays of objects in Python. How can they be created and utilized effectively?

An array of objects is a list of instances.

```
class Student:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
students = [Student("Alice"), Student("Bob")]
```

```
for s in students:
```

```
    print(s.name)
```

